

Tech Stack Finalization

AI-Powered FDA Regulatory Intelligence Platform (Solo-Founder, Accuracy-First, LangGraph + Python)

Constraints applied throughout:

- **Priority:** Accuracy & trust (best retrieval quality)
- **Deployment:** Hybrid — cloud + optional on-prem
- **Graphiti role:** Enhance RAG (hybrid, not replacement)
- **Team:** Solo founder — minimize moving parts

Recommended Stack

**LangGraph + Qdrant
+ BGE-M3 + Claude
API**

Optional: OpenSearch (BM25),
Graphiti+Neo4j at V1

Graphiti Introduction

V1 (Month 6+)

Not MVP — introduced after hybrid
RAG is stable

Est. Monthly Infra (MVP)

\$300–\$1,500

Qdrant + FastAPI + Claude API for
10 pilot users

Time to MVP

4–6 months solo

With Graphiti added at Month 6–9
for cross-guidance reasoning

Executive Recommendation (Read This First)

Final Architecture

Hybrid RAG (Qdrant + OpenSearch + BGE Reranker) orchestrated by LangGraph, with Graphiti (Neo4j) introduced in V1 for cross-guidance reasoning and temporal fact tracking.

What to Avoid Solo

Do not start with Graphiti as your primary retrieval. It requires Neo4j 5.26+ operations, LLM-based entity extraction (adds cost + latency), and adds a second database to babysit. Layer it on later.

Why LangGraph over pure LangChain/ LlamaIndex

LangGraph gives you **explicit stateful nodes with conditional edges** — exactly what regulated Q&A needs (refuse-on-low-confidence, retry-on-failed-guardrail, retrieval routing). LlamaIndex remains best for **ingestion + retrieval primitives** and is used *inside* LangGraph nodes.

Why Graphiti (not vanilla GraphRAG)

Graphiti is **incremental, bi-temporal, and sub-second** — versus Microsoft's GraphRAG which is batch-oriented and slow. Bi-temporal edges (valid_at / invalid_at) map directly to FDA's Draft → Final → Withdrawn lifecycle.

[2](#)

1. Production-Ready Open-Source-First Tech Stack

1.1 Recommended Stack (MVP + V1)

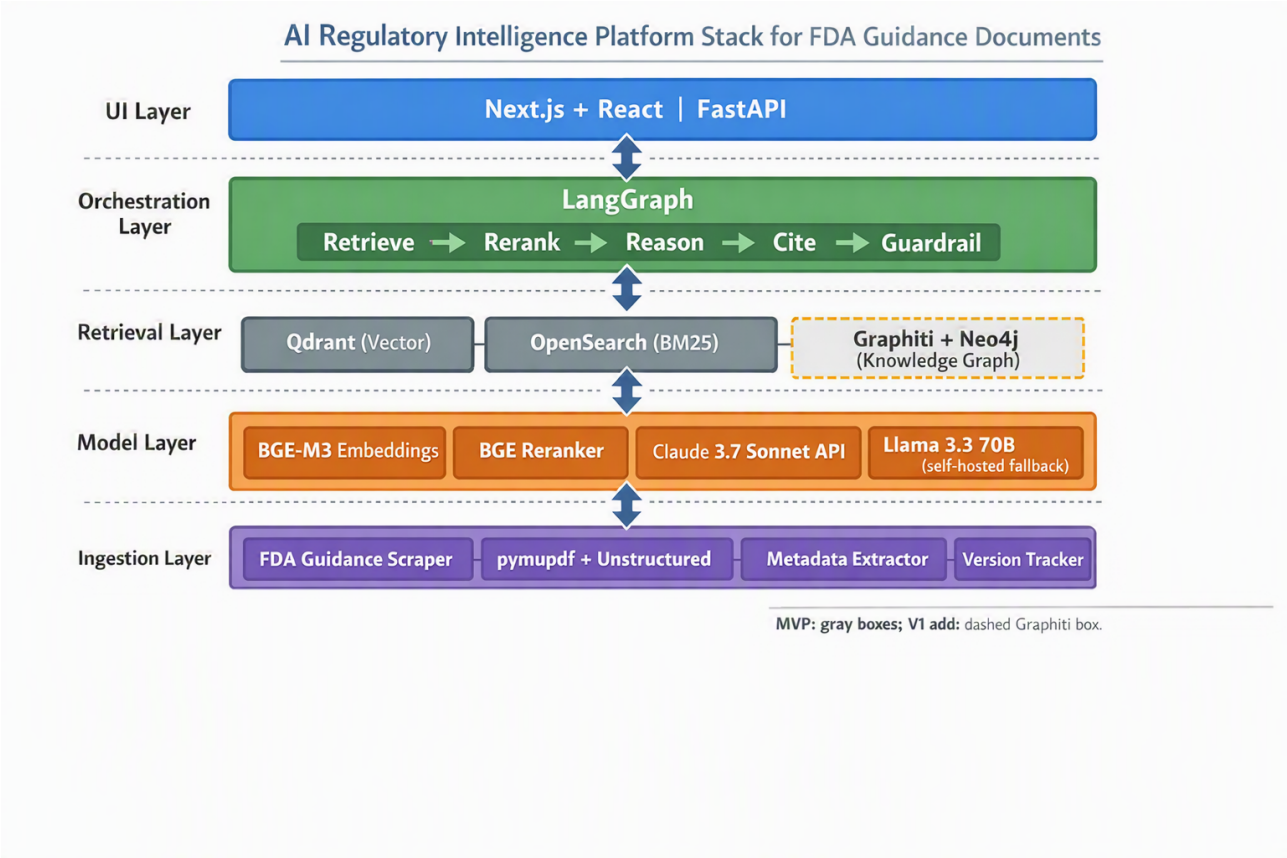


Figure 1: Recommended layered open-source-first tech stack. MVP components in solid boxes; Graphiti knowledge graph (dashed) is introduced in V1 as an enhancement over the hybrid RAG core.

1.2 Layer-by-Layer Choices with Rationale

Layer	Component	Choice	Why (solo + accuracy)
Language / runtime	Python 3.11	Standard	LangGraph + LlamaIndex + Graphiti are all Python-first 2

Layer	Component	Choice	Why (solo + accuracy)
Orchestration	LangGraph 0.2+	✓ Primary	Stateful graph orchestration, conditional edges, built-in persistence & streaming 7
Retrieval primitives	LlamaIndex	✓ Inside nodes	Best data connectors + parent-child chunking; use as a library, not as the top-level orchestrator
Vector DB	Qdrant (self-hosted)	✓ Primary	Fast, simple Docker deploy, filtered hybrid search, Apache 2.0
Keyword / BM25	OpenSearch (or Qdrant native BM25)	✓	Full-text and metadata filtering; on-prem-friendly
Knowledge Graph (V1)	Graphiti (getzep) + Neo4j 5.26	✓ V1	Bi-temporal, incremental, hybrid semantic+keyword+graph search 2

Layer	Component	Choice	Why (solo + accuracy)
Embeddings	BGE-M3 (BAAI)	✓	Top MTEB performance; multilingual; self-hostable
Reranker	BGE-Reranker-Large (cross-encoder)	✓	Precision jump on top-20 → top-5
LLM (MVP)	Claude 3.7 Sonnet API	✓	Highest citation-grounding quality; fast; cheap enough for pilots
LLM (on-prem fallback)	Llama 3.3 70B or Qwen 3 72B via vLLM	✓	Same interface via OpenAI-compatible endpoint
PDF ingestion	<code>pymupdf</code> + <code>unstructured</code>	✓	Structure-preserving parsing; free
API framework	FastAPI + Uvicorn	✓	Async, typed, standard for Python AI stacks
Frontend	Next.js 15 + React	✓	Solo-friendly; good streaming UX
Auth / Users	Supabase or Clerk	✓	Solo-friendly; SOC 2 shortcut

Layer	Component	Choice	Why (solo + accuracy)
Observability	Langfuse (self-hosted or cloud)	✓	Free tier, LangGraph-native tracing
Evaluation	RAGAS + Vectara HHEM	✓	Faithfulness + hallucination scoring on gold set
Deployment	Docker + Docker Compose (MVP), K8s (V2)	✓	Single-node Docker Compose is enough for pilots

Key principle for a solo founder: every component in this stack has a **single self-hosted Docker image** and can also run as a managed service if you want to trade money for time.

2. Why LangGraph — Deep Analysis

2.1 Why LangGraph Over LangChain or LlamaIndex for This Use Case

LangGraph is the **graph-based orchestration layer of the LangChain ecosystem**, purpose-built for combining LangChain components into production-ready applications with persistence, streaming, and other production features [7](#). For a **regulated, citation-first Q&A workflow**, that's exactly the shape of the problem:

- **Stateful nodes with typed state** — you carry `query`, `retrieved_chunks`, `citations`, `confidence`, `guardrail_result` across the graph.
- **Conditional edges** — "if confidence < threshold, refuse" is a *first-class primitive* in LangGraph.
- **Retry loops** — regenerate answer if faithfulness guardrail fails, with counter and max attempts.

- **Streaming + persistence** — checkpoints let you resume long-running agentic runs and stream tokens to the UI.
- **Framework maturity** — LangGraph now defines production RAG alongside LlamaIndex and Haystack.

2.2 LangGraph vs LlamaIndex vs LangChain — Where Each Shines

Framework	Best At	Weakness for our use case	Verdict
LangGraph	Stateful multi-step agents, conditional flow, retries, refuse logic	Ingestion primitives are thinner	✅ Orchestrator
LlamaIndex	Best data connectors, parent-child chunking, retrieval primitives (40% faster retrieval reported vs LangChain in benchmarks)	Less flexible for complex agent flows	✅ Retriever library inside nodes
LangChain (core)	Building blocks; big ecosystem	Rapid API churn; not ideal top-level orchestrator any more	⚠️ Use only for integrations
Haystack	Production pipelines, enterprise search	Smaller community	⚠️ Optional if you value stability over ecosystem

Framework	Best At	Weakness for our use case	Verdict
DSPy	Prompt optimization	Learning curve; overkill for MVP	✗ Defer to V1

Solo-founder decision: LangGraph = orchestration; LlamaIndex = retrieval helpers inside LangGraph nodes; ignore Haystack/DSPy for MVP.

2.3 LangGraph Node/Edge Design for FDA RAG

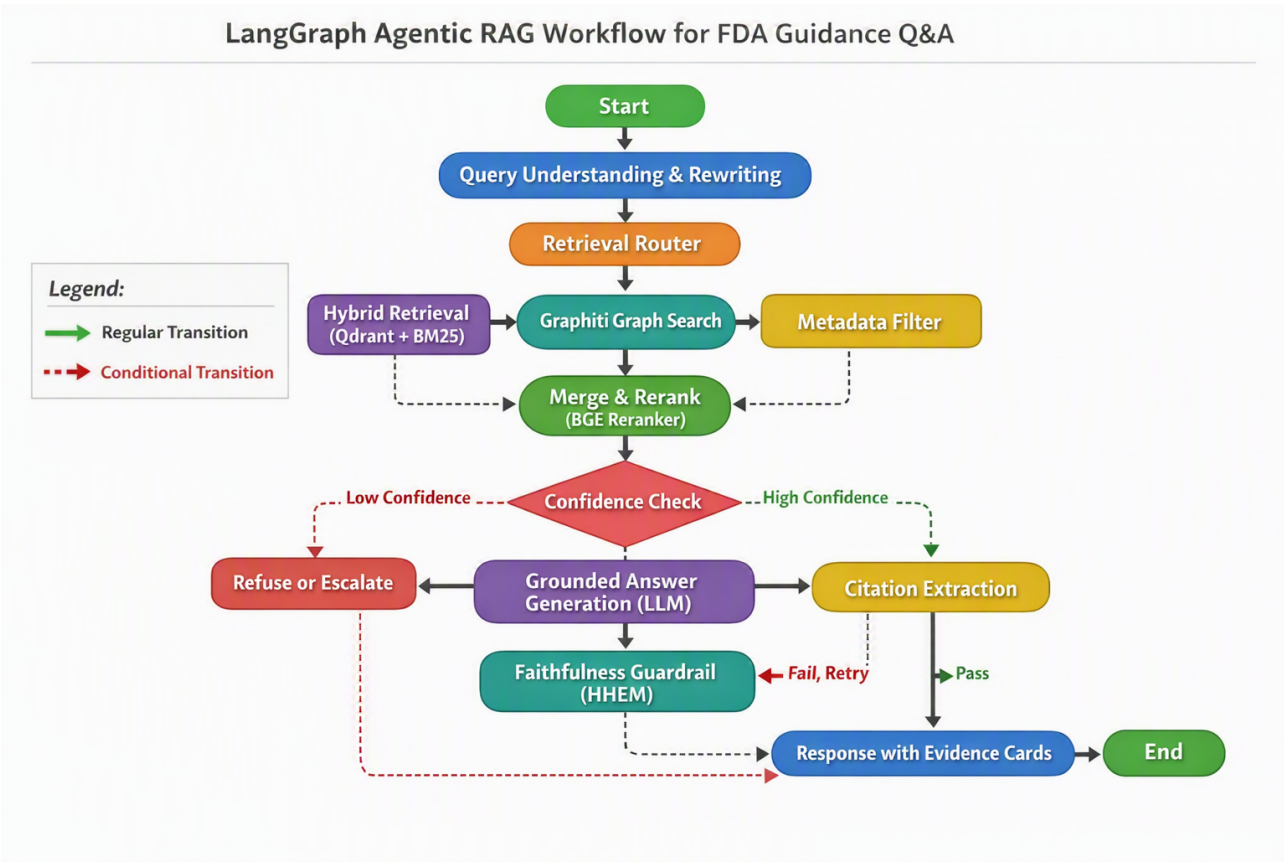


Figure 2: LangGraph node/edge design for the FDA regulatory RAG workflow, including a retrieval router (hybrid + Graphiti + metadata), rerank, faithfulness guardrail, and refuse-on-low-confidence path.

2.4 The Ten Nodes You Actually Need

#	Node	Responsibility	Key libs
1	<code>query_understand</code>	Rewrite question, extract entities (drug, device, CFR), classify intent	LlamaIndex + Pydantic
2	<code>retrieval_router</code>	Decide which retrievers to call (hybrid always; graph only for cross-guidance questions)	LangGraph conditional edge
3	<code>hybrid_retrieve</code>	Qdrant dense + OpenSearch BM25 + metadata filter, reciprocal rank fusion	Qdrant, OpenSearch, LlamaIndex
4	<code>graph_retrieve</code> (V1)	Graphiti <code>search()</code> + <code>get_related_entities()</code>	graphiti-core
5	<code>merge_rerank</code>	Concatenate candidates → BGE cross-encoder → top-5	sentence-transformers
6	<code>confidence_check</code>	If top-1 reranker score < 0.35 → refuse path	Pydantic

#	Node	Responsibility	Key libs
7	<code>generate_answer</code>	Citation-first prompt, LLM call, structured output (<code>answer</code> , <code>citations[]</code>)	Anthropic SDK / OpenAI-compat
8	<code>citation_bind</code>	Attach <code>doc_id</code> , <code>section</code> , <code>passage</code> , <code>version_hash</code> , <code>verify_url</code> to each <code>[n]</code>	Custom
9	<code>faithfulness_guard</code>	Vectara HHEM or RAGAS faithfulness check → retry if fail	ragas, vectara-hhem
10	<code>respond</code>	Emit final Evidence-Card payload to UI	FastAPI SSE

3. Deep Analysis of Graphiti (getzep/graphiti)

3.1 What Graphiti Actually Is

Graphiti is an **Apache 2.0-licensed Python framework for building and querying temporal context graphs for AI agents**, currently at version 0.29.2 (released June 2026), with 28.7k+ GitHub stars [2](#). It sits on top of **Neo4j 5.26+** (or FalkorDB 1.1.2+) as the graph store [6](#).

Unlike static knowledge graphs, Graphiti's context graphs **track how facts change over time, maintain provenance to source data, and support both prescribed and learned ontology** — making them purpose-built for agents operating on evolving, real-world data [2](#).

Unlike traditional RAG, Graphiti **continuously integrates user interactions, structured and unstructured enterprise data, and external information into a coherent, queryable graph**. The framework supports **incremental data updates, efficient retrieval, and precise historical queries without requiring complete graph recomputation** [2](#).

3.2 Core Data Model

A Graphiti context graph contains four primitives [2](#):

Component	Stores
Entities (nodes)	People, products, policies, concepts — with summaries that evolve over time
Facts / Relationships (edges)	Triplets (Entity → Relationship → Entity) with temporal validity windows
Episodes (provenance)	Raw data as ingested — the ground truth stream. Every derived fact traces back here
Custom Types (ontology)	Developer-defined entity and edge types via Pydantic models

3.3 Graphiti's Six Key Capabilities

- 1. Temporal Fact Management** — Facts have validity windows; when info changes, old facts are **invalidated, not deleted**. Query "what's true now" or "what was true at time T" [2](#).
- 2. Episodes & Provenance** — Every entity/relationship traces back to raw source data [2](#).

3. **Prescribed & Learned Ontology** — Define entity/edge types via Pydantic (prescribed), or let structure emerge (learned) [2](#).
4. **Incremental Graph Construction** — New data integrates immediately, no batch recomputation [2](#).
5. **Hybrid Retrieval** — Combines semantic embeddings, BM25 keyword, and graph traversal — sub-second latency, no LLM summarization required [2](#).
6. **Scalability** — Parallel processing, pluggable graph backends [2](#).

3.4 Graphiti vs Microsoft GraphRAG

Graphiti explicitly positions itself as different from Microsoft's GraphRAG [2](#):

Aspect	Microsoft GraphRAG	Graphiti
Primary Use	Static document summarization	Dynamic, evolving context for agents
Data Handling	Batch-oriented	Continuous incremental updates
Knowledge Structure	Entity clusters & community summaries	Temporal context graph with validity windows
Retrieval	Sequential LLM summarization	Hybrid semantic + keyword + graph search
Adaptability	Low	High
Temporal Handling	Basic timestamps	Explicit bi-temporal tracking with automatic fact invalidation
Contradiction Handling	LLM-driven summarization	Automatic fact invalidation, temporal history preserved

Aspect	Microsoft GraphRAG	Graphiti
Query Latency	Seconds to tens of seconds	Sub-second

Why this matters for FDA: Draft → Revised Draft → Final → Withdrawn is *exactly* the pattern Graphiti's bi-temporal edges solve. Microsoft GraphRAG would require full re-summarization on each corpus refresh — Graphiti just invalidates the old edge and adds a new one.

3.5 Graphiti — Strengths and Limitations for FDA Guidance

 Strengths for FDA Use Case

1) Bi-temporal edges map naturally to draft/final/withdrawn lifecycle. 2) Provenance to source episodes = citations. 3) Sub-second latency vs GraphRAG. 4) Apache 2.0 + Python native. 5) Active development (v0.29 as of June 2026).

 Limitations for Solo Founder

1) Requires **Neo4j 5.26+** — adds a second database. 2) LLM calls for entity/relationship extraction add cost + latency at ingest. 3) Custom retrieval logic required (Zep's managed service takes it further). 4) No pre-built citation UX — you build it. 5) Steeper learning curve than a vanilla vector DB.

 Production Readiness

Graphiti powers Zep's production context graphs at sub-200 ms latency at scale. The OSS core version requires you to run the surrounding system yourself — **self-managed only**. For a solo founder, this is doable but non-trivial.

 Cost Consideration

Graphiti calls an LLM during ingestion for entity/fact extraction. For the FDA corpus (2,788 documents), estimate 3–10× the ingestion LLM cost of pure vector RAG. Use a cheap model (Gemini Flash or Qwen) as the Graphiti LLM to control cost.

4. Analysis of the ottomator Graphiti Implementations

Cole Medin's **ottomator-agents** repo contains **two relevant implementations** — the small `graphiti-agent` demo and the more comprehensive `agentic-rag-knowledge-graph` flagship.

4.1 `graphiti-agent` (The Demo You Linked)

Structure ⁵:

- `quickstart.py` — comprehensive Graphiti features tutorial
- `agent.py` — conversational Pydantic AI agent that queries the Graphiti graph
- `llm_evolution.py` — simulation showing how facts evolve over time across three phases
- `requirements.txt` — dependencies

Prerequisites ⁵:

- Python 3.10+
- Neo4j 5.26+
- OpenAI API key

Key insight: This is a **learning demo**, not a production template. It shows *how* Graphiti + Pydantic AI works, but does not implement:

- Document ingestion pipeline
- Hybrid vector + BM25 retrieval
- Reranking
- Citation UX
- Faithfulness guardrails

Reusable for FDA platform?  The `agent.py` pattern (Pydantic AI + Graphiti tool functions) is directly reusable inside a LangGraph node. But you should **not** base your MVP on this demo alone.

4.2 **agentic-rag-knowledge-graph** — The One You Should Actually Study

This is the **flagship implementation** in ottomotor-agents (importance score 19.91) that combines traditional RAG with Graphiti temporal knowledge graph [1](#). It is the closest reference implementation to what you should build for FDA.

Architecture — Dual Database [1](#):

- **PostgreSQL + pgvector** — for vector similarity search over document chunks
- **Neo4j + Graphiti** — for temporal knowledge graph over extracted entities/facts

Agent tools exposed to the Pydantic AI agent [1](#):

- `vector_search` — semantic search over chunks
- `graph_search` — semantic graph search over facts
- `hybrid_search` — combines vector + BM25
- `get_document` — fetch full document
- `list_documents` — enumerate corpus
- `get_entity_relationships` — traverse graph
- `get_entity_timeline` — temporal fact retrieval

GraphitiClient design [4](#):

- Init with LLM config (OpenAI/OpenRouter/Ollama/Gemini)
- Embedder (OpenAIEmbedder)
- Cross-Encoder/Reranker (OpenAIRerankerClient)
- Neo4j driver (bolt://localhost:7687)
- Operations: `build_indices_and_constraints()`, `add_episode()`, `search()`, `get_related_entities()`, `get_entity_timeline()`, `clear_graph()`

Knowledge graph capabilities called out [4](#):

- Temporal facts with `valid_at` and `invalid_at` timestamps
- Automatic entity extraction (companies, technologies, people, locations)
- Episode-based storage where documents are ingested as episodes Graphiti processes
- Semantic graph search using embeddings

Verdict — how to use this for FDA:

What to Reuse	What to Adapt	What to Skip
✓ Dual-database pattern (Postgres/pgvector + Neo4j/Graphiti)	🔧 Replace Pydantic AI with LangGraph for stateful workflows and refuse-on-low-confidence	✗ Postgres/pgvector — use Qdrant instead for filtered hybrid search speed at 100M+ vec scale
✓ Agent tool signatures (7 tools)	🔧 Replace entity types (companies, tech) with FDA-specific (guidance, docket, CFR reference, product code, drug/device)	✗ Don't ship Graphiti in MVP — start with hybrid RAG only
✓ Episode-based ingestion	🔧 Add FDA-specific parent-child chunking, citation bindings	✗ Skip the entity_timeline tool until V1

5. Comparison — Three Retrieval Architectures for FDA

5.1 Head-to-Head Scoring

Dimension (10 = best)	A. Pure Hybrid RAG	B. Hybrid RAG + Graphiti (Recommended)	C. Full GraphRAG
Accuracy (single-hop Q&A)	8	9	7

Dimension (10 = best)	A. Pure Hybrid RAG	B. Hybrid RAG + Graphiti (Recommended)	C. Full GraphRAG
Accuracy (multi-hop / cross-guidance)	5	9	9
Explainability	9	9	8
Complexity (solo founder)	9	6	3
Cost (infra + LLM)	9	6	4
Time to MVP	9 (4-6 mo)	6 (6-9 mo)	3 (9-12 mo)
Operational risk	9	6	4
Temporal reasoning (draft vs final)	5	10	6
Weighted total	63	60	44

*Note: A scores higher on the weighted total in the pure numbers, but B unlocks unique capabilities (temporal reasoning, cross-guidance) that A cannot match. The right decision is **A for MVP, then evolve to B.***

5.2 Written Trade-Off

- **Option A (Pure Hybrid RAG)** — Ship in 4–6 months solo; 90–95% recall@10 achievable on FDA; excellent for single-guidance Q&A. **Fails** on questions like "How did the FDA's stance on real-world evidence evolve between 2019 and 2024?" or "Which guidances cross-reference 21 CFR 312.23?".

- **Option B (Hybrid RAG + Graphiti)** — Adds a second database (Neo4j) and Graphiti ingestion. Unlocks cross-guidance reasoning, temporal drift ("what did guidance say in Q3 2024?"), and entity-centric queries. Cost: 6–9 months solo; higher ingest cost.
- **Option C (Full GraphRAG)** — Entity extraction dominates; retrieval is slow (seconds to tens of seconds) ²; requires re-summarization when corpus updates. **Not appropriate for a solo founder with a live-updating FDA corpus.**

5.3 Verdict

MVP (Months 0–6)

Option A — Pure Hybrid RAG. Ship citation-first Q&A, prove trust, sign 5–10 pilots.

V1 (Months 6–12)

Option B — Add Graphiti layer for cross-guidance comparison, temporal drift, and entity-centric queries. LangGraph *routes* to graph search only for graph-shaped questions.

Never

Option C (batch GraphRAG) is a poor fit for a corpus that updates daily and a solo founder who cannot re-run overnight jobs safely.

6. Final Recommendation

6.1 MVP Architecture (Ship This First)

- **Orchestration:** LangGraph (10-node graph per Section 2.4)
- **Retrieval:** Qdrant (dense) + OpenSearch or Qdrant native BM25 (sparse) + BGE-Reranker
- **Ingestion:** Python + `pymupdf` + `unstructured` + Prefect (or plain APScheduler for solo)
- **LLM:** Claude 3.7 Sonnet API (MVP); provide Llama 3.3 70B via vLLM as a config-flag fallback for on-prem customers
- **Embeddings:** BGE-M3 (self-hosted with `sentence-transformers` or via HF Text-Embedding-Inference container)

- **UI:** Next.js 15 + shadcn/ui + Vercel AI SDK for streaming
- **API:** FastAPI + Pydantic v2 + SSE for streaming tokens
- **Auth:** Supabase (SSO, RBAC free tier)
- **Storage:** Postgres (Supabase) for metadata, sessions, audit trail; S3-compatible (R2/MinIO) for original PDFs
- **Observability:** Langfuse
- **Evaluation:** RAGAS + Vectara HHEM + a hand-curated FDA gold set (200 questions)
- **Deployment:** Single VM + Docker Compose (Hetzner/DigitalOcean for cost); moved to Fly.io/AWS ECS at ~20 customers

6.2 Where Graphiti Belongs

- **NOT in MVP.**
- **YES in V1 (Month 6+).**
- Deploy **Neo4j 5.26 Community Edition** [3](#) alongside Qdrant.
- Ingest each FDA guidance as a Graphiti **episode**, using **prescribed Pydantic ontology** (custom entity types: `Guidance`, `Docket`, `CFRReference`, `ProductCode`, `Center`, `TopicArea`; custom edges: `SUPERSEDES`, `CITES`, `APPLIES_TO`, `REVISED_FROM`).
- Add a `graph_retrieve` node to LangGraph, invoked only when the intent router classifies a query as **cross-guidance, temporal, or entity-centric**.
- Use the Zep-managed offering only if OSS operational load becomes a burden — Graphiti's OSS version is self-hosted only [2](#).

6.3 What to Explicitly Avoid as Solo Founder

❌ Full GraphRAG

Batch summarization does not fit a daily-updated FDA corpus; too slow.

❌ Two orchestrators

Do not run LangChain *and* LangGraph *and* LlamaIndex agent workflows in parallel. Pick LangGraph as the orchestrator; use LlamaIndex retrievers as helpers.

✗ Fine-tuning at MVP

BGE-M3 out of the box is enough for pilots; defer domain fine-tuning to V1 after you have real user query logs.

✗ K8s from day one

Docker Compose on one VM is enough for 10 pilots. Migrate later when infra pain justifies it.

✗ Building your own reranker

Use BGE-Reranker-Large off the shelf.

✗ Multi-LLM router at MVP

One frontier LLM (Claude 3.7) + one OSS fallback (Llama 3.3 70B) is enough. Add complexity only when a customer pays for it.

2

6.4 Migration Path — MVP → Advanced

Milestone	Add	Reason
M0–3 (PoC)	LangGraph + Qdrant + BGE + Claude	Prove hybrid RAG works on FDA corpus
M3–6 (MVP)	OpenSearch (BM25), Reranker, Faithfulness Guard, Evidence Cards, Langfuse	Ship citation-first product to first pilots
M6–9 (V1)	Neo4j + Graphiti; add <code>graph_retrieve</code> node; cross-guidance comparison feature	Unlock temporal + relationship reasoning

Milestone	Add	Reason
M9–15 (V2)	vLLM Llama 3.3 for on-prem edition; multi-tenant; Azure OpenAI edition for BAA	Enterprise deployment
M15+ (V3)	Fine-tune BGE-M3 on FDA corpus; DSPy prompt optimization; agentic multi-hop	Squeeze accuracy further

7. Implementation Blueprint

7.1 Repository Structure

```
fda-regulatory-copilot/
├─ pyproject.toml
├─ docker-compose.yml          # Qdrant, OpenSearch, Neo4j (V1), Langfuse, App
├─ .env.example
├─ README.md
├─
├─ apps/
│  └─ api/                    # FastAPI app
│     │ └─ main.py
│     │ └─ routes/
│     │    │ └─ chat.py
│     │    │ └─ search.py
│     │    │ └─ documents.py
│     │    └─ health.py
│     └─ schemas/             # Pydantic v2 models (Answer, EvidenceCard, ...)
│     └─ deps.py              # DI: LLM, embedder, qdrant, opensearch, graphiti
│     └─ settings.py          # Pydantic Settings
│     └─
│  └─ web/                    # Next.js 15 + shadcn/ui (optional at MVP)
│  └─
├─ src/
│  └─ ingestion/
│     │ └─ fda_scraper.py      # Guidance-search page + PDF URLs
│     │ └─ pdf_parser.py       # pymupdf + unstructured wrapper
│     │ └─ metadata_extractor.py # docket, center, topic, status, dates
│     │ └─ chunking.py         # parent-child (section-paragraph)
│     │ └─ embedder.py         # BGE-M3 client
│     │ └─ version_tracker.py  # SHA-256 content hash + supersedes graph
│     └─ pipeline.py          # Prefect/APScheduler flow
```

```

| |
| | └─ retrieval/
| |   │ └─ qdrant_client.py
| |   │ └─ opensearch_client.py
| |   │ └─ hybrid.py           # Reciprocal Rank Fusion
| |   │ └─ reranker.py         # BGE-Reranker cross-encoder
| |   │ └─ filters.py          # center, date, status filters
| |
| | └─ graph/                  # V1
| |   │ └─ graphiti_client.py  # wraps graphiti-core
| |   │ └─ ontology.py         # Pydantic entity/edge types
| |   │ └─ episode_builder.py  # FDA doc → Graphiti episode
| |   │ └─ graph_search.py     # search + get_related + timeline
| |
| | └─ llm/
| |   │ └─ router.py           # Claude vs Llama vs Azure OpenAI
| |   │ └─ prompts.py         # citation-first templates
| |   │ └─ streaming.py
| |
| | └─ agents/
| |   │ └─ state.py            # LangGraph typed state
| |   │ └─ nodes.py           # 10 nodes from Section 2.4
| |   │ └─ edges.py           # conditional routing logic
| |   │ └─ guardrails.py       # HHEM + confidence check
| |   │ └─ citation.py         # bind [n] → EvidenceCard
| |   │ └─ graph.py           # StateGraph builder
| |
| | └─ eval/
| |   │ └─ golden_set.json     # 200 curated FDA Q&A
| |   │ └─ ragas_eval.py
| |   │ └─ hhem_check.py
| |
| | └─ common/
| |   │ └─ logging.py
| |   │ └─ telemetry.py        # Langfuse tracing
| |   │ └─ errors.py
| |
| └─ data/
|   │ └─ raw_pdfs/             # S3/MinIO in prod
|   │ └─ parsed/               # cached parsed docs
|   │ └─ snapshots/           # daily version snapshots
|
| └─ infra/
|   │ └─ docker/
|   │   │ └─ qdrant.Dockerfile
|   │   │ └─ opensearch.Dockerfile
|   │   │ └─ neo4j.Dockerfile  # V1
|   │   │ └─ app.Dockerfile
|   │   └─ k8s/                # later
|
| └─ tests/
|   │ └─ unit/
|   │ └─ integration/
|   │ └─ eval/                 # nightly quality regression

```

7.2 Minimal Viable Pipeline — Step-by-Step Data Flow

Step	Component	What happens
1. Discover	<code>fda_scraper.py</code>	Daily crawl of FDA guidance search page; enumerate new/updated docs
2. Fetch	<code>fda_scraper.py</code>	Download PDF + landing HTML; store to <code>data/raw_pdfs/</code> with SHA-256
3. Parse	<code>pdf_parser.py</code>	<code>pymupdf</code> → text + structure; <code>unstructured</code> → tables; store <code>data/parsed/{doc_id}.json</code>
4. Extract metadata	<code>metadata_extractor.py</code>	Fill <code>{doc_id, title, center, topic, status, issue_date, docket, url_pdf}</code>
5. Version	<code>version_tracker.py</code>	If hash new → snapshot; if hash changed → record <code>supersedes</code> edge
6. Chunk	<code>chunking.py</code>	Parent = section (~1500 tok), child = paragraph (~300 tok) with 15% overlap
7. Embed	<code>embedder.py</code>	BGE-M3 dense vectors for child chunks

Step	Component	What happens
8. Index	<code>qdrant_client.py</code> + <code>opensearch_client.py</code>	Upsert vectors + BM25 index with all metadata for filtering
9. (V1) Graph ingest	<code>graph/episode_builder.py</code>	Wrap each doc as Graphiti episode; LLM extracts entities (Guidance, Docket, CFR, ProductCode) and edges (SUPERSEDES, CITES)
10. Query	<code>agents/graph.py</code> (LangGraph)	Q → understand → route → hybrid retrieve (+ optional graph retrieve) → rerank → confidence → generate → citation bind → faithfulness → respond
11. Trace	<code>common/telemetry.py</code>	Langfuse captures spans; nightly eval regression on gold set

7.3 Reference `docker-compose.yml` (MVP)

```

services:
  api:
    build: ./apps/api
    depends_on: [qdrant, opensearch, langfuse]
    env_file: .env
    ports: ["8000:8000"]

  qdrant:
    image: qdrant/qdrant:latest
    volumes: [qdrant_data:/qdrant/storage]
    ports: ["6333:6333"]

  opensearch:
    image: opensearchproject/opensearch:2

```

```

environment:
  - discovery.type=single-node
  - plugins.security.disabled=true
volumes: [os_data:/usr/share/opensearch/data]
ports: ["9200:9200"]

langfuse:
  image: langfuse/langfuse:latest
  ports: ["3000:3000"]

# ---- V1 additions (uncomment when adding Graphiti) ----
# neo4j:
#   image: neo4j:5.26-community
#   environment:
#     - NEO4J_AUTH=neo4j/${NEO4J_PASSWORD}
#   volumes: [neo4j_data:/data]
#   ports: ["7474:7474", "7687:7687"]

volumes:
  qdrant_data:
  os_data:
  # neo4j_data:

```

7.4 Reference `requirements.txt` (Python Deps)

```

# Orchestration
langgraph>=0.2.60
langchain-core>=0.3
langchain-anthropic>=0.3
langchain-openai>=0.3

# Retrieval primitives
llama-index-core>=0.12
llama-index-vector-stores-qdrant>=0.4
llama-index-embeddings-huggingface>=0.4

# Vector + BM25
qdrant-client>=1.11
opensearch-py>=2.7

# Embeddings + reranker
sentence-transformers>=3.2
BGE-M3 model via HF (no separate pkg)

# Ingestion
pymupdf>=1.24
unstructured[pdf]>=0.15
beautifulsoup4>=4.12
httpx>=0.27
prefect>=3.0 # or apscheduler for lighter footprint

# API + UI backend
fastapi>=0.115

```

```

uvicorn[standard]>=0.32
pydantic>=2.9
pydantic-settings>=2.6
sse-starlette>=2.1

# Auth / storage
supabase>=2.9 # optional
boto3>=1.35    # S3 / R2 / MinIO
psycopgg[binary]>=3.2

# Observability + eval
langfuse>=2.50
ragas>=0.2
opentelemetry-sdk>=1.27

# V1 additions
# graphiti-core>=0.29
# neo4j>=5.26

```

7.5 First Working Prototype — Two Weeks Solo

Day	Deliverable
1–2	Repo scaffold, docker-compose up with Qdrant + OpenSearch + FastAPI hello-world
3–4	FDA scraper for 50 sample guidances; PDF parse; metadata extract
5–6	Parent-child chunking; BGE-M3 embeddings; index into Qdrant with metadata
7	BM25 index in OpenSearch; hybrid RRF fusion
8	BGE-Reranker top-20 → top-5
9–10	LangGraph 5-node graph (understand → retrieve → rerank → confidence → answer) with Claude 3.7 Sonnet

Day	Deliverable
11	Evidence Card schema + citation binding
12	Vectara HHEM guardrail node + retry edge
13	Langfuse tracing; 30-question gold set; RAGAS eval
14	Streaming UI with 3 test users

End state after 2 weeks: working prototype answering FDA guidance questions with citations, running on a single \$40/mo Hetzner VM.

8. Bottom Line

The Answer

LangGraph + Qdrant + BGE + Claude API for MVP; add Graphiti (Neo4j) in V1. This gives you accuracy, trust, hybrid deployment, and solo-founder feasibility.

Study This Reference

The **ottomator** **agentic-rag-knowledge-graph** project is the closest working reference to what you should build. Study its dual-database pattern and agent tool signatures, but replace Postgres/pgvector with Qdrant and Pydantic AI with LangGraph.



The Trap to Avoid

Do not fall into **architecture astronautics**. Graphiti is exciting technology, but it is *not* your MVP. Ship citation-first hybrid RAG in 4–6 months, then add Graphiti when the product is trusted.



The One Non-Negotiable

Every answer must ship with a verifiable citation from day one. This is your only real moat against Veeva, Cortellis, and every ChatGPT wrapper. Guard it with a faithfulness check and a refuse-on-low-confidence node — *even in the MVP*.

1

If you want next: I can generate a downloadable `.md` file of this analysis, a `docker-compose.yml` + `pyproject.toml` starter you can `git clone`, or a **GitHub Copilot-ready** `CLAUDE.md` / `.github/copilot-instructions.md` so Copilot writes code aligned to this architecture from day one.